2. Problem Statement Content

2.1 Business Context

A mid-sized insurance company sells Whole Life insurance products through an agent-assisted digital order entry platform. The platform is used by licensed insurance agents who collect information from clients and submit policy applications digitally instead of relying on paper-based forms or manual back-office entry.

The platform supports the following primary agent journeys:

- User authentication and secure access: Agents log in securely, complete required authentication checks, and access their dashboard.
- Client and insured information capture: Agents enter personal details, date of birth, address, contact details, KYC identification, financial suitability information, and compliance declarations.
- Insurance product selection: Agents browse Whole Life products, review eligibility, and select a suitable product for the insured client.
- Policy configuration and rider selection: Agents configure face value, premium frequency, payment term, policy term, and applicable riders.
- Beneficiary management: Agents add primary and secondary beneficiaries, capture beneficiary details, define allocation percentages, and provide guardian details for minor beneficiaries.
- Document upload and validation: Agents upload required KYC documents and resolve validation errors before submission.
- Application submission and status tracking: Agents save drafts, resume incomplete applications, submit valid applications, receive application reference numbers, and track application status.

2.2 Business Problem

The company has observed that regression testing is becoming slow and inconsistent as new features are added to the insurance order entry platform. Manual QA cycles are unable to fully cover login, client data capture, KYC validation, product selection, rider business rules, beneficiary allocation, document upload, application submission, API validations, and negative scenarios before every release.

Some examples of issues include:

- Valid agents are unable to log in after authentication changes.
- Locked agents receive incorrect or misleading error messages.
- Password policy validations are not consistently enforced.
- Client age eligibility rules are missed.
- Future date of birth is accepted.
- Mandatory KYC documents are not enforced before submission.
- SSN or sensitive identification data is not masked properly.
- Income versus premium affordability validation is not applied.
- Product eligibility is not validated against client age or financial suitability.
- Rider coverage is allowed to exceed base policy face value.
- Mutually exclusive waiver riders can both be selected.
- Premium calculation does not update after policy configuration changes.
- Beneficiary allocation below or above 100% is allowed.

- Minor beneficiary guardian details are not required.
- Duplicate beneficiaries are not blocked.
- Draft application data is not persisted correctly.
- Submit button is enabled even when validation errors exist.
- Application status on dashboard does not match backend status.
- API validation differs from UI validation.
- Test failures are difficult to debug because screenshots, traces, and reports are not captured consistently.

The business problem is therefore not only about writing Playwright scripts. It is about building a professional QA automation approach that improves release confidence, reduces repetitive manual regression effort, and gives technical and business stakeholders clear evidence of application quality.

2.3 Project Objectives

The objective of this project is to design and implement a Playwright-based QA automation solution that validates critical agent journeys, API behavior, regression coverage, and reporting outputs for the insurance order entry platform.

- Establish end-to-end automation coverage for high-risk insurance journeys such as login, client information entry, Whole Life product selection, rider configuration, beneficiary setup, document upload, validation, submission, and status tracking.
- Design a structured automation framework using Playwright with JavaScript or TypeScript, preferably TypeScript for stronger maintainability.
- Implement Page Object Model so that test logic is readable, reusable, and easier to maintain.
- Validate UI behavior across forms, dynamic sections, tables, modals, document upload controls, review screens, validation summaries, and confirmation screens.
- Automate positive, negative, boundary, and regression scenarios for insurance application workflows.
- Include API tests for authentication, client data, products, riders, applications, validations, submissions, and application status.
- Compare API data with UI output wherever applicable to detect integration issues.
- Configure CI/CD execution so that the test suite can run automatically on code push, pull request, schedule, or manual trigger.
- Use debugging tools such as Playwright trace viewer, screenshots, videos, network logs, and console logs.
- Generate professional test reports and defect logs that mentors and business stakeholders can review.
- Apply automation best practices such as stable locators, reusable fixtures, clean test data, meaningful assertions, and avoiding hardcoded waits.

2.4 Application Flow Overview

The application flow starts with agent authentication and moves through insurance application creation, product configuration, beneficiary setup, validation, submission, and post-submission tracking.

Step	Flow	QA Focus
1	Login / Authentication	Credential validation, password rules, account lockout, MFA/OTP simulation, session handling, error messages, unauthorized access prevention.
2	Agent Dashboard	Draft applications, submitted applications, status cards, search, filters, resume draft, start new application.
3	Client / Insured Information	Personal details, DOB and age calculation, gender, marital status, occupation, employer details, mandatory fields.
4	Address and Contact Information	Residential address, mailing address, same-as-residential behavior, email validation, phone validation, preferred communication mode.
5	KYC, Financial, and Compliance Details	ID type, ID number validation, SSN masking, document upload, income, net worth, source of funds, risk profile, PEP declaration.
6	Product Selection	Whole Life product listing, eligibility, product details, face value, premium frequency, payment term, policy term.
7	Rider Selection	Rider categories, age eligibility, coverage limits, dependency rules, maximum rider count, mutually exclusive rider rules.
8	Beneficiary Management	Primary and secondary beneficiaries, allocation percentage, minor beneficiary guardian details, duplicate prevention, beneficiary type-specific fields.
9	Document Upload and Validation	Accepted formats, file size, mandatory document checks, upload confirmation, missing-document errors.
10	Submission and Status Tracking	Validation summary, submit button control, application reference number, read-only submitted application, dashboard status, API/UI consistency.

2.5 QA Lifecycle Expected

Enterprise QA automation requires more than scripting. Learners are expected to follow a disciplined QA lifecycle.

Study
Study the business process, user roles, application modules, API endpoints, test data, browser expectations, validation rules, and release risks. Understand how an insurance application moves from UI entry to backend record and user-visible status.

Plan
Prepare a QA automation strategy that defines the scope, out-of-scope items, test types, browser coverage, test data needs, API scope, regression tagging, reporting approach, and risk-based priorities.

Design
Convert the QA plan into detailed test cases. Each test case should define the module, scenario, preconditions, test steps, expected result, priority, test type, automation status, and evidence expectation.

Automate
Build a Playwright automation framework using Page Object Model, fixtures, reusable utilities, environment configuration, API helpers, test data files, and meaningful assertions.

Execute
Run the automation suite locally and through CI/CD. Capture passed tests, failed tests, skipped tests, screenshots, traces, videos, console logs, and execution reports.

Analyze
Analyze failures and determine whether they are application defects, automation defects, test data issues, environment issues, or unstable tests. Explain the business impact of important failures.

Report
Submit execution reports, defect logs, evidence, final QA summary, and a project walkthrough that can be understood by both technical mentors and business stakeholders.

2.6 Technology Environment

The project environment should simulate a real enterprise web application and test automation setup. The application can be hosted in a shared lab environment, with learners accessing it through a browser and running Playwright scripts from a hosted workspace or local development setup.

Component	Purpose
Insurance order entry demo web application	Provides the agent-facing UI for client information, product selection, rider configuration, beneficiaries, document upload, validation, and submission.
Backend APIs	Expose authentication, products, riders, applications, validations, submissions, and status tracking.
Test database / seed data	Provides predefined agents, clients, products, riders, applications, documents, statuses, and seeded defects.

Hosted learner workspace	Allows learners to write, run, debug, and share Playwright automation work with mentors.
Playwright test framework	Executes UI, API, regression, smoke, validation, and E2E tests.
Git repository	Stores test automation code, README, reports, and CI/CD configuration.
GitHub Actions or equivalent CI/CD	Runs tests automatically and stores reports/artifacts.
Reporting artifacts	Playwright HTML reports, screenshots, traces, videos, logs, and defect evidence.

2.7 Expected Outcome

At the end of the project, learners should have built and executed a QA automation suite for the insurance order entry platform with the following deliverables.

- Study and QA automation plan.
- Test scenario inventory and detailed test cases.
- Playwright automation framework with UI tests.
- API test suite using Playwright request context or equivalent helper.
- Page Object Model implementation.
- Test data and configuration files.
- Playwright HTML report and execution logs.
- Defect log with evidence.
- Final QA summary and project walkthrough.

Technical Outcomes

- Use Playwright with JavaScript or TypeScript.
- Automate browser-based E2E journeys.
- Create reusable Page Objects and fixtures.
- Run API validations and response schema checks.
- Use test tags for smoke, regression, API, E2E, authentication, client, product, rider, beneficiary, and workflow tests.
- Configure screenshots, videos, traces, retries, and reports.
- Execute tests through CI/CD and upload artifacts.

QA Outcomes

- Understand business risk before automation.
- Design meaningful positive and negative test cases.
- Prioritize high-value regression scenarios.
- Identify defects with clear evidence and impact.
- Analyze flaky tests and environment issues separately from application defects.
- Communicate findings professionally.

Business Outcomes

- Improved confidence in insurance application submission workflows.
- Reduced manual regression effort.
- Faster issue detection before releases.